

Sistema Diagnostico Avanzato per Stampa 3D: Un Approccio Integrato ML + OntoBK

Componenti del gruppo:

- Leonetti Antonio, [MAT. 802614],
a.leonetti26@studenti.uniba.it

Link GitHub:
[ICON4-25-26](#)

A.A. 2025 - 2026

Sommario

Capitolo 1 – Introduzione	3
1.1 Dominio di interesse	3
1.2 Sommario del KBS	3
1.3 Requisiti funzionali e strumenti	3
1.4 Argomenti di interesse	4
Capitolo 2 – Rappresentazione della Conoscenza (Ontologie).....	5
2.1 Ingegnerizzazione dell'Ontologia e Scelte Implementative.....	5
2.2 Definizione della TBox: Tassonomia delle Classi e Gerarchie Profonde	5
2.3 Object Properties: Relazioni, Transitività e Inverse	6
2.4 Restrizioni Logiche ed Assiomi (Description Logic).....	7
2.5 Definizione della ABox: Istanze, Topologia e Sintomi Grezzi	8
Capitolo 3 – Ragionamento Automatico	9
3.1 Il Paradigma del Ragionamento Automatico nel KBS	9
3.2 Il Motore di Inferenza HermiT e le Logiche Descrittive	9
3.3 Implementazione del Modulo semantic_reasoner.py	10
3.4 Meccanismo di Materializzazione dei Sintomi ed Estrazione Classi Indirette	11
3.5 Ottimizzazione delle Prestazioni: Il Sistema di Caching Semantico	12
Capitolo 4 – Apprendimento e Incertezza (OntoBK)	13
4.1 Incertezza nel Dominio e Modellazione Multimodale dei Dati.....	13
4.2 Il Paradigma OntoBK e il Semantic Lifting	14
4.3 Pre-processing dello Spazio delle Feature: Standardizzazione e Codifica Nominale .	15
4.4 Il Modello Predittivo Ensemble: Random Forest	16
4.5 Classificazione a Massimizzazione del Margine: Support Vector Machines (SVM)	16
4.6 Il Paradigma Conessionista: Reti Neurali Artificiali (Multi-Layer Perceptron).....	16
4.7 Valutazione Statistica Rigorosa e Analisi Comparativa delle Prestazioni	17
4.8 L'Agente Adattativo: Apprendimento Online e Feedback Loop	18
Capitolo 5 – Conclusioni e Sviluppi Futuri	19
5.1 Sintesi dei Risultati Raggiunti	19
5.2 Valutazione Critica dell'Architettura	19
5.3 Sviluppi Futuri.....	20

Capitolo 1 – Introduzione

1.1 Dominio di interesse

Il dominio di questo progetto è la diagnostica dei guasti nelle stampanti 3D a deposizione di filamento fuso (FDM). Si tratta di un ambito caratterizzato da un'elevata complessità dovuta all'interazione tra componenti meccaniche (cinghie, motori), termiche (estrusore, piatto) ed elettroniche (sensori, scheda madre). Risolvere un'anomalia richiede spesso un'analisi che superi la semplice osservazione di un sintomo isolato, rendendo necessaria l'integrazione di conoscenza strutturata per disambiguare le cause dei malfunzionamenti.

1.2 Sommario del KBS

Il progetto realizza un **Knowledge-Based System (KBS)** progettato come un agente computazionale capace di agire in modo intelligente per risolvere task diagnostici. Il sistema integra tre moduli principali:

1. **Modulo di Rappresentazione:** Utilizza un'ontologia formale (OWL 2.0) per modellare la gerarchia dei componenti e le loro relazioni logiche.
2. **Modulo di Ragionamento:** Sfrutta un reasoner a Logica Descrittiva (HermiT) per inferire conoscenza implicita (es. dedurre categorie di allarme non esplicite).
3. **Modulo di Apprendimento:** Implementa un modello di Machine Learning (Random Forest) arricchito tramite **Ontology Background Knowledge (OntoBK)**. In questo approccio, il sistema non apprende solo dai dati grezzi (sintomi), ma utilizza le inferenze prodotte dall'ontologia come "conoscenza di fondo" per migliorare l'accuratezza predittiva.

1.3 Requisiti funzionali e strumenti

Il sistema è stato interamente sviluppato in **Python**, linguaggio scelto per la sua versatilità e per la disponibilità di librerie standard. I requisiti funzionali del KBS sono stati soddisfatti attraverso l'integrazione dei seguenti strumenti tecnici:

- **Owlready2:** È la libreria core per la gestione della **Knowledge Base**. Permette di caricare e manipolare l'ontologia OWL 2.0, gestendo classi, proprietà e individui come oggetti Python. Include l'integrazione con il reasoner **HermiT**, essenziale per la fase di ragionamento automatico (Inference) e per la materializzazione della conoscenza implicita.
- **Scikit-learn:** Utilizzata per il modulo di **Apprendimento**. Fornisce l'implementazione dell'algoritmo **Random Forest** e gli strumenti per la validazione statistica del sistema, come la funzione `cross_validate` e la classe `RepeatedKFold`, necessarie per garantire una valutazione scientifica e non basata su singoli run.

- **Pandas:** Impiegata per la manipolazione efficiente del dataset diagnostico. Permette di gestire i dati strutturati (sintomi e guasti) estratti dal file CSV e di trasformarli in formati compatibili con gli algoritmi di apprendimento.
- **NumPy:** Fondamentale per la gestione dei vettori numerici (feature vectors). Viene utilizzata per convertire le osservazioni semantiche in dati computabili dal modello di Machine Learning e per il calcolo delle metriche statistiche (medie e deviazioni standard).

1.4 Argomenti di interesse

Il presente elaborato affronta le seguenti tematiche:

1. **Rappresentazione della Conoscenza (Logiche Descrittive):** Modellazione formale del dominio della stampante 3D tramite un'ontologia espressa in OWL 2.0. La base di conoscenza (KB) è strutturata formalmente dividendola in **TBox** (assiomi sui concetti, relazioni transitive e inverse) e **ABox** (asserzioni sulle istanze reali e sui sintomi grezzi).
2. **Ragionamento Automatico (Meccanismi di Inferenza):** Utilizzo del logician reasoner standard **HermiT** per eseguire la materializzazione della conoscenza implicita. Il sistema non effettua pattern matching algoritmico, ma applica la deduzione logica per riclassificare i sintomi e ottimizza il processo mediante un sistema personalizzato di memoria cache.
3. **Apprendimento Supervisionato e Gestione dell'Incertezza:** Implementazione di un classificatore ad albero ensemble (Random Forest) basato sul paradigma **OntoBK (Ontology Background Knowledge)**. Il Machine Learning apprende da vettori numerici arricchiti semanticamente (*Semantic Lifting*). La stabilità del modello è convalidata tramite una procedura rigorosa di *Repeated K-Fold Cross-Validation*, affiancata da un ciclo di feedback per l'apprendimento online in tempo reale.

Capitolo 2 – Rappresentazione della Conoscenza (Ontologie)

2.1 Ingegnerizzazione dell'Ontologia e Scelte Implementative

La conoscenza può essere definita come informazione stabile e a lungo termine su un dato dominio; tuttavia, per permettere al sistema di operarvi dei calcoli, essa va necessariamente rappresentata e formalizzata in termini di un rigoroso linguaggio di rappresentazione. La Base di Conoscenza (KB) si configura così come la rappresentazione interna all'agente intelligente, e le sue proprietà fondanti devono garantire sia la ricchezza espressiva per modellare il dominio, sia la trattabilità per consentire un'elaborazione efficiente.

Nel sistema diagnostico in considerazione, è stato scartato l'uso dell'ontologia come mero database (tassonomia passiva), adottando invece il linguaggio standard OWL 2.0. Utilizzando la libreria Python owlready2 all'interno del modulo `ontology_core.py`, il mondo della stampante 3D è stato modellato descrivendo gli "Individui" (le entità fisiche della stampante), raggruppandoli in "Classi" concettuali e vincolandoli attraverso "Proprietà" che rappresentano le relazioni oggettuali (Object Properties).

2.2 Definizione della TBox: Tassonomia delle Classi e Gerarchie Profonde

La Terminological Box (TBox) contiene le astrazioni del nostro dominio, strutturate in modo gerarchico. Teoricamente, una classe è definita come un insieme di individui che ne costituiscono le istanze, sia effettive che potenziali. Nell'ontologia costruita, è stato istituito un albero semantico che deriva da una super-classe radice denominata `EntitaDiagnostica`, che eredita nativamente dal concetto universale `Thing` di OWL.

Al fine di differenziare entità strutturali da manifestazioni di malfunzionamento, `EntitaDiagnostica` si dirama in due concetti ortogonali: la classe `Componente` e la classe `Sintomo`. La gerarchia viene formata sfruttando la nozione di inclusione: un concetto `S` è sottoclasse di `C` se ogni individuo di tipo `S` è anche di tipo `C`. Conseguentemente, per riflettere le diverse nature ingegneristiche della macchina, `Componente` è stato specializzato in:

- `ComponenteElettrico`
- `ComponenteMeccanico`
- `ComponenteTermico`

Un aspetto implementativo avanzato, vitale per il successivo ragionamento semantico, è l'utilizzo dell'**ereditarietà multipla**, applicata per modellare entità fisicamente ibride. Nel codice di `ontology_core.py` è stato infatti modellato:

- `MotoreStepper(ComponenteMeccanico, ComponenteElettrico)`: un motore fonde la meccanica dei movimenti con la natura elettrica dell'assorbimento di corrente.
- `Termistore(ComponenteTermico, ComponenteElettrico)`: il sensore che legge la temperatura traducendola in segnale elettrico.

- **Ugello**(ComponenteMeccanico, ComponenteTermico): l'estremità finale dove il movimento meccanico del filamento subisce il processo di fusione termica.

Questa modellazione permette al sistema di non essere confinato ad una rigida gerarchia ad albero, modellando invece un DAG (Grafo Aciclico Orientato) in cui un malfunzionamento al termistore verrà automaticamente e semanticamente percepito dal sistema in entrambe le dimensioni (elettrica e termica).

Per consentire al sistema di associare i dati fisici dei sensori a concetti logici astratti, la TBox è stata arricchita con macro-gerarchie radicate in Thing:

- **StatoOperativo:** Mappa il contesto dinamico e ambientale della macchina. Si articola nelle sottoclassi disgiunte AmbienteUmido (per alti tassi di umidità), AltaVelocita (per sollecitazioni cinematiche intense) e UsuraAvanzata (per identificare condizioni limite di invecchiamento hardware).
- **CategoriaDegrado:** Modella i profili di vulnerabilità fisica dei componenti meccatronici, suddividendosi in DegradoMeccanico (inerte a cinghie e motori) e DegradoTermico (inerte a ugelli e termistori).
- **Sensore:** Definita come sottoclasse di ComponenteElettrico, rappresenta l'hardware dedicato al monitoraggio di specifici parametri fisici. Include sottoclassi specializzate come l'**Accelerometro** (per il monitoraggio delle risonanze cinematiche) e ri-classifica moduli già esistenti come il Termistore (che eredita ora in modo multiplo da ComponenteTermico e Sensore).
- **TipoCalibrazione:** Modella i task di manutenzione prescrittiva richiesti per il ripristino delle condizioni nominali della macchina. Si articola in specializzazioni come CalibrazionePID (per il controllo termico) e CalibrazioneStep (per la cinematica).

2.3 Object Properties: Relazioni, Transitività e Inverse

Le interdipendenze tra le classi non sono sufficienti se non esplicitamente relazionate. Le proprietà rappresentano relazioni binarie per le quali è ingegneristicamente essenziale specificare un *dominio* (le classi dei soggetti) e un *codominio* o range (le classi degli oggetti).

Nel modulo `ontology_core.py` sono state definite tre fondamentali Object Properties:

1. **ha_sottocomponente e parte_di:** Regolano la scomposizione strutturale e cartesiana della stampante. `ha_sottocomponente` è definita come `TransitiveProperty` (Proprietà Transitiva); l'assioma garantisce che se l'asse X ha un sottocomponente e questo a sua volta include un sotto-modulo, l'intero sotto-modulo è implicitamente legato all'asse originario. `parte_di` è dichiarata come l'esatta proprietà inversa (`inverse_property`), operando un linking bidirezionale automatico tra ABox e TBox.
2. **coinvolge_componente:** Collega la classe epifenomenica Sintomo alla classe fisica Componente, fungendo da ponte fondamentale per il *Semantic Lifting*.
3. **sensibile_a:** Stabilisce un vincolo causale tra le entità diagnostiche e gli stati ambientali critici. Mappa il comportamento differenziale di alcuni materiali ad alte prestazioni (es. *NYLON* o *PETG*) rispetto a perturbazioni esterne.

4. **aggravato_da**: Relaziona le manifestazioni dei sintomi a specifiche dinamiche cinematiche o temporali di stress della macchina (come l'elevata velocità di estrusione o il superamento delle ore di ciclo vita dell'hardware).
5. **alimentato_da**: Modella esplicitamente l'albero delle dipendenze energetiche e funzionali meccatroniche. Permette al sistema di mappare come un'anomalia di un nodo primario (es. *SchedaMadrePrincipale*) si propaghi a cascata su tutti i moduli elettrici ad esso asserviti.
6. **conseguenza_di**: Formalizza le catene di causalità diretta tra i sintomi stessi. È configurata come *TransitiveProperty* per abilitare la *Root Cause Analysis* (RCA). Se un sintomo A è conseguenza di un sintomo B, e B è conseguenza di un'ostruzione C, il Reasoner dedurrà autonomamente la catena $A \rightarrow C$.
7. **soggetto_a_degrado**: Mappa la vulnerabilità intrinseca dell'hardware associando ciascun componente al rispettivo profilo di usura fisica.
8. **misurato_da**: Costituisce il collante semantico tra il dominio logico-epifenomenico e quello hardware-telemetrico. Collega la manifestazione di un guasto testuale (Sintomo) allo specifico hardware deputato al suo monitoraggio (Sensore), istruendo il sistema su quale strato fisico interrogare in caso di anomalia.
9. **richiede_calibrazione**: Introduce la semantica della manutenzione prescrittiva. Non si limita a diagnosticare il guasto, ma associa un Componente compromesso alla specifica *TipoCalibrazione* necessaria per risolverlo, fornendo una direttiva operativa diretta (*actionable insight*).

2.4 Restrizioni Logiche ed Assiomi (Description Logic)

Affinché l'ontologia funga da autentica Background Knowledge (BK) per il Machine Learning (OntoBK) e non da semplice schema dati, essa deve definire assiomi, ossia proposizioni vere nell'interpretazione intesa dal progettista. L'assiomatizzazione del dominio applicativo permette di riclassificare implicitamente gli individui.

Nel progetto, tale principio si sostanzia nelle *Restrizioni Logiche* (*Description Logic*) espresse mediante il costrutto `equivalent_to`. Piuttosto che scrivere regole rigide imperanti la classificazione di un guasto, abbiamo definito tre macro-categorie astratte di sintomi: *AllarmeTermico*, *AllarmeMeccanico* e *AllarmeElettrico*.

Guardando direttamente al codice, un allarme termico è definito formalmente come:

- `equivalent_to = [Sintomo & coinvolge_componente.some(ComponenteTermico)]`

Questa espressione logica impone al KBS una ferrea regola deduttiva: qualsiasi istanza appartenente alla classe *Sintomo* che abbia la proprietà `coinvolge_componente` legata a *qualunque* istanza discendente dalla classe *ComponenteTermico* (anche qualora l'entità fosse ibrida, come il *Termistore*), deve essere categorizzata a runtime come *AllarmeTermico*. Questa implementazione sfrutta la *Description Logic* pura per elevare i dati grezzi a vettori semantici (*Semantic Lifting*), abilitando il reasoning trattato nel capitolo successivo.

Ulteriori restrizioni logiche considerate sono:

- **AllarmeAdesioneIgroskopica**: Identifica una classe di allarmi complessi generati dall'interazione tra sintomi fisici e alterazioni microclimatiche, definita formale come un qualsiasi sintomo la cui manifestazione sia aggravata da una condizione di *AmbienteUmido*.

- **AllarmeCriticoUsura:** Cattura lo stato di invecchiamento hardware deducendo automaticamente la criticità strutturale quando un sintomo si manifesta in concomitanza di un quadro di UsuraAvanzata.

2.5 Definizione della ABox: Istanze, Topologia e Sintomi Grezzi

La Assertional Box (ABox) popola la Knowledge Base fornendo la definizione estensionale, ossia l'elenco e la configurazione pratica delle sue istanze nel dominio materiale. In `ontology_core.py` è stata costruita l'esatta architettura della stampante 3D istanziando prima i singoli componenti fisici (ad es. `mainboard = SchedaMadre("SchedaMadrePrincipale")`, `motore_x`, `ugello_1`, `termistore_hotend`, `piatto`, `cinghia_x`, ecc.) e i profili di contesto operativo o di degrado (come `ContestoAmbienteUmido` o `ProfiloDegradoMeccanico`).

La connessione tra le parti sfrutta le Object Properties precedentemente definite per assemblare un albero topologico e funzionale multidimensionale, operando su quattro livelli di asserzione:

1. **Topologia Meccanica:** Viene strutturata la scomposizione cartesiana della macchina popolando, ad esempio, l'array transitivo dell'estrusore tramite la relazione:
`extruder_asm.ha_sottocomponente = [motore_e, termistore_hotend, ugello_1]`.
2. **Dipendenza Meccatronica (Albero Elettrico):** Viene mappata la distribuzione energetica legando i componenti hardware alla scheda madre attraverso la proprietà `alimentato_da` (es. `motore_x.alimentato_da = [mainboard]`), consentendo di modellare la propagazione dei guasti elettrici.
3. **Profili di Invecchiamento (Vulnerabilità):** I componenti soggetti a usura vengono associati alle rispettive categorie di deterioramento mediante la proprietà `soggetto_a_degrado` (es. `cinghia_x.soggetto_a_degrado = [degrado_mecc]`).
4. **Reti Sensoriali e Manutenzione Proattiva:** Vengono istanziati i sensori hardware avanzati (come `sensore_vib = Accelerometro(...)`) e le routine di ripristino. Attraverso la proprietà `richiede_calibrazione`, l'ontologia mappa le soluzioni manutentive sui componenti fisici (es. `motore_x.richiede_calibrazione = [cal_step]`). Parallelamente, i sintomi vengono legati ai sensori che ne intercettano la firma fisica (`s_layer_sp.misurato_da = [sensore_vib]`).

L'ultimo passo della rappresentazione riguarda l'inserimento dei sintomi grezzi. Essi costituiscono il vocabolario diagnostico base – le vere entità che il software cattura in fase di monitoraggio – e vengono correlati sia all'hardware tramite la proprietà `coinvolge_componente`, sia al contesto operativo mediante la proprietà `aggravato_da`. Inoltre, la proprietà transitiva `conseguenza_di` permette di concatenare i sintomi in una rete causale per la *Root Cause Analysis*. Alcuni esempi significativi definiti nel codice includono:

```
Sintomo("TempTroppoAlta", coinvolge_componente=[termistore_hotend])
```

```
Sintomo("LayerSpostati", coinvolge_componente=[motore_x, cinghia_x],  
      aggravato_da=[config_veloce, config_usura])
```



```
Sintomo("Warping", coinvolge_componente=[piatto], aggravato_da=[config_umido])
```

```
# Modellazione della catena causale dei sintomi
```

```
s_ticchettio.conseguenza_di = [s_no_filam]
```

```
s_sotto_est.conseguenza_di = [s_ticchettio]
```

Tale mappatura asserzionale rappresenta i fatti primitivi del dominio di interesse. Sarà poi il motore logico (il Reasoner HermiT) ad attivarsi su questa ABox, estraendo le relazioni implicite e traducendo le osservazioni testuali e telemetriche in vettori astratti semantici ad alto livello concettuale. Questa operazione di *Semantic Lifting* garantisce all'infrastruttura multi-algoritmica del Machine Learning (Random Forest, SVM e reti neurali MLP) un dato ad altissima densità informativa ed esplicabilità (XAI), eliminando i rigidi vincoli delle tradizionali analisi condizionali statiche.

Capitolo 3 – Ragionamento Automatico

3.1 Il Paradigma del Ragionamento Automatico nel KBS

Il nucleo operativo che distingue un Knowledge-Based System (KBS) avanzato da un comune software procedurale risiede nella sua capacità di astrazione ed inferenza. Nella progettazione del sistema diagnostico per la stampante 3D, l'inserimento di fatti espliciti all'interno della ABox non esaurisce la conoscenza disponibile; al contrario, costituisce il punto di partenza per l'attivazione dei meccanismi di ragionamento automatico.

L'inferenza è il processo computazionale attraverso il quale viene generata nuova conoscenza (fatti o relazioni prima impliciti) a partire da una base di conoscenza esistente, applicando regole deduttive formalmente valide.

Nel contesto della diagnostica industriale, un approccio puramente algoritmico basato su catene di istruzioni condizionali soffre di una drammatica rigidità: l'aggiunta di un nuovo sensore o la riclassificazione di un guasto richiederebbe la riscrittura della logica di controllo del software. Nell'agente intelligente in considerazione, l'indipendenza della conoscenza dalla logica di controllo viene preservata demandando la strutturazione deduttiva a un motore di ragionamento (Reasoner) logico-simbolico. Il modulo `semantic_reasoner.py` si assume il compito di interrogare la base di conoscenza, valutare la coerenza dei modelli e materializzare le relazioni latenti indotte dalle definizioni assiomatiche della TBox, fornendo al successivo strato predittivo una rappresentazione logicamente solida e arricchita.

3.2 Il Motore di Inferenza HermiT e le Logiche Descrittive

Per l'esecuzione dei compiti inferenziali sul modello OWL 2.0 sviluppato, è stato integrato il modulo di ragionamento logico standard HermiT, interfacciato nativamente mediante i costrutti della libreria `owlready2` attraverso la funzione `sync_reasoner`. Da un punto di vista teorico, HermiT si basa sul framework delle Logiche Descrittive (DL) e, nello specifico, implementa l'algoritmo di *Hypertableaux*. Questo formalismo garantisce proprietà matematiche fondamentali per la stabilità del KBS: la correttezza (soundness), ovvero la garanzia che ogni fatto dedotto sia logicamente implicato dalla KB, e la completezza

(completeness), che assicura che tutte le implicazioni logiche valide vengano effettivamente individuate dal motore senza omissioni.

Un aspetto teorico cardine che ha guidato l'integrazione di HermiT riguarda l'Assunzione di Mondo Aperto (Open World Assumption - OWA), tipica del Web Semantico e dei linguaggi derivati da OWL. A differenza dei sistemi basati su database relazionali tradizionali (che operano sotto *Closed World Assumption* - CWA, dove ciò che non è esplicitamente memorizzato è considerato falso), l'OWA stabilisce che la mancanza di un'informazione non equivale alla sua negazione, ma semplicemente a uno stato di non-conoscenza. HermiT sfrutta questo paradigma per operare in regime di monotonicità logica: l'introduzione di nuovi fatti o nuove istanze nella ABox non invalida le deduzioni precedentemente effettuate, ma espande la frontiera della conoscenza esplicita. L'algoritmo di Hypertableaux riduce lo spazio di ricerca delle interpretazioni possibili minimizzando la generazione di modelli ridondanti, traducendo le restrizioni espresse in Description Logic (come le restrizioni esistenziali *some* o le intersezioni *&* descritte nel Capitolo 2) in vincoli computazionali stringenti e decidibili.

3.3 Implementazione del Modulo `semantic_reasoner.py`

Nel codice del progetto, l'astrazione del motore inferenziale è centralizzata nella classe `DiagnosticReasoner`. Il costruttore riceve l'istanza dell'ontologia e inizializza le strutture dati di controllo:

```
class DiagnosticReasoner:

    def __init__(self, ontology):

        self.onto = ontology

        self.reasoner_run = False

        self.cache_inferenze = {} # <-- MEMORIA CACHE VELOCE
```

La computazione ha inizio con l'invocazione del metodo `run_inference()`. Questo metodo apre il contesto dell'ontologia in memoria e lancia l'esecuzione del motore HermiT:

```
def run_inference(self):

    print("[Reasoner] Avvio di HermiT DL Reasoner...")

    with self.onto:

        sync_reasoner(infer_property_values=True)

    self.reasoner_run = True

    print("[Reasoner] Inferenza completata. Nuove classi materializzate.")
```

Il parametro `infer_property_values=True` indica esplicitamente al configuratore di non limitarsi a verificare la consistenza tassonomica delle classi, ma di procedere alla *materializzazione* dei valori delle proprietà e delle appartenenze individuali. Durante questa fase, HermiT esamina le asserzioni della ABox alla luce delle definizioni logiche della TBox.

Ad esempio, analizzando l'istanza Sintomo("TempTroppoAlta") — associata tramite coinvolge_componente al componente termistore_hotend (il quale è, per via dell'ereditarietà gerarchica, un ComponenteTermico).

3.4 Meccanismo di Materializzazione dei Sintomi ed Estrazione Classi Indirette

Una volta completata la computazione globale dell'algoritmo di Hypertableaux, la base di conoscenza si trova in uno stato "materializzato". Il software deve poter estrarre questa conoscenza implicita per renderla disponibile alle fasi successive di vettorizzazione del Machine Learning. Tale compito è assolto dal metodo `get_inferred_classes_for_symptom(symptom_name)`:

```
def get_inferred_classes_for_symptom(self, symptom_name):
    if not self.reasoner_run:
        raise RuntimeError("Eseguire run_inference() prima di interrogare il reasoner.")
    # Se il sintomo è stato già calcolato lo si pesca direttamente dalla cache
    if symptom_name in self.cache_inferenze:
        return self.cache_inferenze[symptom_name]
    istanza_sintomo = self.onto.search_one(iri=f"*{symptom_name}")
    if not istanza_sintomo:
        return []
    inferred = [cls.name for cls in istanza_sintomo.INDIRECT_is_a
                if hasattr(cls, 'name') and cls.name not in ["Thing", "Sintomo",
                                                            "EntitaDiagnostica"]]
    # salvataggio nella cache
    self.cache_inferenze[symptom_name] = inferred
    return inferred
```

Il passaggio ingegneristico fondamentale in questo frammento di codice è l'utilizzo dell'attributo `.INDIRECT_is_a` fornito da `owlready2`. Invece di interrogare l'attributo standard `.is_a` (che restituirebbe unicamente le classi esplicitamente dichiarate in fase di design della ABox, riducendo l'ontologia a una tabella passiva), `.INDIRECT_is_a` interroga il grafo semantico ristrutturato dal Reasoner.

Questo attributo estrae tutte le super-classi e le classi equivalenti indotte per via logica, risalendo l'intero reticolo delle descrizioni. Il codice applica poi un filtro mirato per escludere i concetti primitivi universali della radice (`Thing`, `Sintomo`, `EntitaDiagnostica`) che, essendo comuni a tutti i flussi, non offrirebbero alcun potere discriminante ai modelli statistici. Il risultato finale restituito è una lista di stringhe rappresentanti i concetti astratti (es. `['AllarmeTermico', 'AllarmeElettrico']` nel caso di entità a coinvolgimento ibrido), traducendo l'evidenza empirica grezza in categorie semantiche formali.

3.5 Ottimizzazione delle Prestazioni: Il Sistema di Caching Semantico

L'interazione tra modelli probabilistici (Machine Learning) e logica matematica (Ontologie) introduce una problematica cruciale legata alla complessità computazionale. Il calcolo delle inferenze tramite algoritmi basati su `Tableaux` presenta una complessità intrinseca intrattabile nel caso peggiore (spesso EXPTIME-complete per logiche espressive). Sebbene la chiamata a `sync_reasoner()` avvenga una sola volta all'avvio del programma principale (fissando lo stato delle inferenze sulla ABox iniziale), la fase di *Semantic Lifting* — ovvero la trasformazione delle righe del dataset grezzo in vettori arricchiti — richiede di scorrere ripetutamente le righe dei casi diagnostici (1000 record nello scenario applicativo in considerazione).

Senza un'adeguata strategia di ottimizzazione software, l'interrogazione iterativa del grafo ontologico tramite `.search_one` e la risoluzione dinamica delle relazioni di ereditarietà indurrebbe un massiccio overhead dovuto alle chiamate inter-linguaggio tra l'interprete Python e il motore semantico sottostante (implementato in Java). Per ovviare a questo vincolo prestazionale, all'interno della classe `DiagnosticReasoner` è stato implementato un **meccanismo di Caching Semantico in memoria**.

Attraverso il dizionario privato `self.cache_inferenze`, la prima volta che un determinato sintomo isolato viene incontrato nel flusso di scansione del dataset, il sistema esegue la ricerca sul grafo, estrae il profilo delle classi indirette e memorizza la corrispondenza chiave-valore (ad esempio, `"TempTroppoAlta" : ["AllarmeTermico"]`). Nelle successive iterazioni, la richiesta viene intercettata immediatamente dal blocco condizionale condotto a tempo costante $O(1)$:

```
if symptom_name in self.cache_inferenze:
    return self.cache_inferenze[symptom_name]
```

Questa scelta architetturale riduce drasticamente i tempi di elaborazione della pipeline: la generazione e il caricamento delle feature semantiche per le 1000 istanze del dataset passano da una sottomissione d'ordine di diversi minuti a una risoluzione quasi istantanea (pochi secondi). Questa ottimizzazione dimostra come considerazioni ingegneristiche sul software possano rendere scalabile l'integrazione della Background Knowledge semantica all'interno di flussi di calcolo statistici intensivi.

Capitolo 4 – Apprendimento e Incertezza (OntoBK)

4.1 Incertezza nel Dominio e Modellazione Multimodale dei Dati

La progettazione di un agente computazionale non può prescindere dalla natura intrinsecamente imperfetta degli ambienti reali, nei quali l'agente opera con percezioni limitate, sensori soggetti a tolleranze ed informazioni incomplete. Nel dominio della diagnostica di macchine complesse come le stampanti 3D FDM, l'ambiguità non risiede solo nel rumore stocastico delle segnalazioni umane, ma anche nell'interazione latente tra lo stato fisico dei componenti e le condizioni operative ambientali.

Per addestrare il sistema a gestire questa incertezza senza ricorrere a banali e fragili alberi decisionali statici, è stato sviluppato il modulo `generate_dataset.py`. Questo script funge da simulatore stocastico dell'ambiente, generando una base dati di 1500 casi diagnostici.

Il dataset non è confinato alla dimensione isolata dei sintomi, ma cattura lo scenario diagnostico attraverso 9 variabili eterogenee, integrando feature categoriche nominali, grandezze fisiche continue e indicatori cinematici:

- **Variabili Cinematiche e di Processo:** La colonna `velocita_stamp` (espressa in mm/s) modella la dinamica dei movimenti cartesiani; velocità elevate introducono sollecitazioni d'inerzia che incrementano probabilisticamente la perdita di passi nei motori passo-passo (correlata al sintomo *LayerSpostati*).
- **Variabili Ambientali e Fisiche:** La colonna `umidita_ambientale` (%) cattura le condizioni termo-igrometriche dell'ambiente di stampa; un'alta umidità agisce da fattore scatenante primario per difetti di estrusione e adesione, degradando per igroscopia tecnopolimeri avanzati come il *NYLON* o il *PETG*.
- **Variabili di Invecchiamento e Consumo:** La colonna `usura_motore` (espressa in ore totali di funzionamento dell'hardware) introduce la dimensione temporale del ciclo di vita dei componenti hardware; superata la soglia critica delle 2000 ore, le distribuzioni di probabilità dei guasti meccanici ed elettrici subiscono un'impennata qualitativa dovuta all'usura naturale di cinghie, cuscinetti e spazzole.
- **Variabili Termiche e di Stato:** Le colonne `temperatura_estrusore` e `temperatura_piatto` registrano i dati telemetrici continui dei sensori, la cui generazione è vincolata condizionalmente alla tipologia di guasto target (es. un valore drasticamente ridotto sul piatto simula un'avaria del circuito o del termistore).

L'iniezione deliberata del 15% di rumore stocastico sui sintomi distribuisce le anomalie al di fuori dei percorsi causali deterministici, costringendo i modelli di Machine Learning a non memorizzare passivamente le combinazioni esatte, ma a perseguire un reale compromesso tra adattamento ai dati e complessità architetturale (ispirato al criterio MAP).

```
def generate_large_dataset(num_samples=1500):
```

```
    # ...
```

```
    for _ in range(num_samples):
```

```
        # ...
```

```

if random.random() < 0.15: # Aumentato il rumore stocastico al 15%
    sintomo_rumore = random.choice(["VentolaRumorosa", "SchermoNero",
    "TicchettioEstrusore"])

    if sintomo_rumore not in sintomi_scelti:
        sintomi_scelti.append(sintomo_rumore)

# Generazione parametrica complessa
# Parametri Base
velocita = random.randint(40, 70)
umidita = random.randint(30, 50)
usura = random.randint(100, 1500)

```

4.2 Il Paradigma OntoBK e il Semantic Lifting

L'approccio più comune, ma metodologicamente debole, per l'analisi di dataset testuali consiste nell'utilizzo di tecniche base di Natural Language Processing (NLP) o in semplici conteggi delle frequenze (Bag of Words) mascherati da apprendimento automatico. Il KBS implementato rifiuta categoricamente questa banalizzazione implementando il paradigma **OntoBK (Ontology Background Knowledge)** all'interno del modulo `ontoBK_learning.py`.

In questo paradigma, l'ontologia descritta nei capitoli precedenti non rimane un'entità astratta, ma diviene vera e propria *Background Knowledge* iniettata nel processo di apprendimento. La classe `SemanticFeatureExtractor` è il cuore di questa trasformazione, eseguendo il cosiddetto *Semantic Lifting*: l'elevazione semantica del dato grezzo. Durante la scansione del dataset, per ogni sintomo testuale (es. "TempTroppoAlta"), il sistema non si limita a registrarne la presenza, ma interroga la cache del Reasoner (HermiT) per estrarne le super-classi logiche inferite.

```

# Fissiamo l'ordine delle feature (fondamentale per il ML)
self.raw_features = sorted(list(self.raw_symptoms_vocab))
self.semantic_features = sorted(list(self.semantic_classes_vocab))

self.feature_names = [f"raw_{s}" for s in self.raw_features] + [f"onto_{c}" for c in
self.semantic_features]

```

Il risultato è un vocabolario dinamico ibrido. Il vettore delle feature finale (feature vector) fornito in pasto al Machine Learning non è composto solo dalle variabili indipendenti grezze (`raw_TempTroppoAlta`, `raw_RumoreCinghia`), ma viene arricchito con dimensioni semantiche ortogonali derivate dalla logica descrittiva (`onto_AllarmeTermico`, `onto_AllarmeMeccanico`). In questo modo, il classificatore statistico dispone di coordinate

astratte per raggruppare istanze che, a livello puramente sintattico, apparirebbero totalmente disgiunte, fondendo così la potenza deduttiva (simbolica) con quella induttiva (statistica).

Per testare la robustezza del paradigma ibrido ML + OntoBK di fronte a falsi positivi diagnostici, lo spazio delle feature numeriche continue comprende il livello_vibrazioni (misurato in Hz e simulato per via stocastica nell'intervallo 5-90 Hz). L'introduzione di questa coordinata telemetrica, coadiuvata dalla proprietà ontologica misurato_da, si è rivelata cruciale per la risoluzione dell'ambiguità logica: un sintomo come la sotto-estrusione, associato a un livello di vibrazioni nominale (<10 Hz), permette alla Rete Neurale di ignorare il falso allarme meccanico del motore e far convergere la probabilità (Softmax) verso una diagnosi di Guasto Termico, dimostrando un comportamento di *cross-validation* interna tra la deduzione logica e la lettura empirica del sensore.

4.3 Pre-processing dello Spazio delle Feature: Standardizzazione e Codifica Nominale

L'introduzione di variabili multimodali (valori continui di temperatura, percentuali di umidità, ore di usura espresse in migliaia e vettori booleani generati dal lifting semantico ontologico) determina una severa disomogeneità nelle scale metriche dello spazio delle feature. Sebbene alcuni modelli ensemble basati su alberi decisionali siano robusti rispetto alla magnitudo delle variabili, le architetture predittive basate sul calcolo delle distanze geometriche (SVM) o sull'ottimizzazione del gradiente (Reti Neurali) subirebbero una distorsione sistematica. Le feature con intervalli numerici più ampi (come le ore di usura_motore o la velocità_stampato) dominerebbero impropriamente la funzione di costo rispetto alle coordinate binarie dell'ontologia (valori 0 o 1).

Per ripristinare la trattabilità computazionale e garantire la convergenza geometrica, la pipeline implementata nel modulo main.py introduce una fase di pre-processing rigorosa:

1. **Codifica One-Hot (One-Hot Encoding):** La variabile categorica nominale materiale, estesa dinamicamente a runtime per mappare l'intero spettro di tecnopolimeri supportati (PLA, ABS, PETG, TPU, NYLON, ASA, PC), viene proiettata in un sottospazio di vettori ortogonali binari (mat_PLA, mat_NYLON, ecc.), neutralizzando qualsiasi arbitraria introduzione di relazioni d'ordine.
2. **Standardizzazione Statistica (StandardScaler):** Ciascun vettore delle feature complessivo X , risultante dalla fusione tra sensori fisici e classi astratte indotte dal Reasoner HermiT, viene sottoposto a una trasformazione lineare che centra la distribuzione sul valore medio ($\mu = 0$) e scala la varianza ($\sigma^2 = 1$), secondo l'operazione:

$$X_{scaled} = \frac{X - \mu}{\sigma}$$

Questo passaggio ingegneristico assicura che il *Semantic Lifting* mantenga inalterata la sua densità informativa, consentendo alle dimensioni logico-simboliche di cooperare equamente con le grandezze empiriche continue dei sensori.

4.4 Il Modello Predittivo Ensemble: Random Forest

Per la fase di classificazione vera e propria, supervisionata dalle etichette di guasto (es. GuastoTermico, ProblemaAdesione), è stato scelto l'algoritmo **Random Forest**, implementato tramite la libreria scikit-learn nel modulo main.py.

La scelta di un modello *ensemble* basato su alberi decisionali multipli risponde a precise necessità ingegneristiche:

1. **Gestione di Spazi Vettoriali Sparsi:** I vettori booleani generati dal *Semantic Lifting* sono altamente sparsi (molti 0 e pochi 1). Le Random Forest gestiscono naturalmente questa tipologia di feature space senza richiedere complesse normalizzazioni necessarie invece per reti neurali o SVM.
2. **Robustezza all'Incertezza:** Mediando (averaging) le predizioni di centinaia di alberi non correlati, il modello abbate la varianza e assorbe in modo eccellente il 10% di rumore introdotto nel dataset, realizzando di fatto quel compromesso tra adattamento ai dati e complessità (criterio MAP).

La mediazione probabilistica su alberi multipli riflette l'approccio bayesiano di averaging su spazi vasti per abbattere la varianza dell'errore.

4.5 Classificazione a Massimizzazione del Margine: Support Vector Machines (SVM)

Al fine di esplorare paradigmi di apprendimento supervisionato alternativi a quello strutturale degli alberi, nel sistema diagnostico è stata integrata un'architettura basata su **Support Vector Machines (SVM)** tramite la classe SVC di scikit-learn. Da un punto di vista teorico, una SVM persegue la sintesi di un iperpiano separatore ottimo capace di massimizzare il margine geometrico tra le zone di decisione delle diverse classi di guasto. I punti del dataset arricchito che si trovano sul confine del margine assumono il ruolo di vettori di supporto (*support vectors*), concentrando su di sé tutta la conoscenza statistica necessaria a definire la frontiera di classificazione.

4.6 Il Paradigma Connessionista: Reti Neurali Artificiali (Multi-Layer Perceptron)

Il vertice della complessità algoritmica del modulo di apprendimento è rappresentato dall'introduzione di un modello connessionista: una **Rete Neurale Artificiale Feedforward** del tipo *Multi-Layer Perceptron* (MLPClassifier). Questa scelta riflette la necessità teorica di disporre di un approssimatore universale di funzioni, in grado di mappare autonomamente rappresentazioni astratte e combinazioni non lineari di feature senza una pre-programmazione esplicita delle interazioni tra le variabili.

L'architettura interna della rete è strutturata su più strati di nodi (neuroni):

- **Input Layer:** Composto da un numero di nodi pari alla dimensionalità dinamica del Feature Vector esteso (somma di sensori fisici, codifiche dei materiali e classi ontologiche).
- **Hidden Layers (Strati Latenti):** Configurato con una topologia profonda a due strati nascosti rispettivamente di 50 e 25 neuroni `hidden_layer_sizes=(50, 25)`. Questa progressiva riduzione della dimensionalità costringe la rete a operare una

compressione informativa, estraendo configurazioni latenti di guasto ad alto livello di astrazione.

- **Funzione di Attivazione non Lineare:** Ciascun neurone degli strati nascosti applica la funzione di rettifica lineare *ReLU* ($f(x) = \max(0, x)$), che garantisce l'introduzione della non-linearità nel sistema evitando al contempo la saturazione del gradiente durante la fase di computazione.
- **Output Layer:** Composto da un numero di nodi pari alle categorie di guasto target (*GuastoTermico*, *GuastoMeccanico*, *ProblemaElettrico*, *ProblemaAdesione*).

L'addestramento della rete avviene mediante l'algoritmo di **Retropropagazione dell'Errore (Backpropagation)**: durante la fase di *forward pass*, l'input fluisce attraverso i pesi sinaptici fino a determinare la classificazione; l'errore commesso rispetto all'etichetta reale viene calcolato tramite una funzione di cross-entropy e propagato a ritroso (*backward pass*), applicando il gradiente stocastico (ottimizzatore Adam) per aggiornare i pesi e minimizzare l'errore di generalizzazione su epoche successive (fissate a un massimo di 300 iterazioni).

4.7 Valutazione Statistica Rigorosa e Analisi Comparativa delle Prestazioni

Per convalidare scientificamente l'efficacia del paradigma *OntoBK* combinato con lo spazio delle feature multimodale, la metodologia di validazione rigetta qualsiasi singolo split empirico, adottando invece una procedura di **Repeated K-Fold Cross-Validation** configurata con $K=5$ split e 5 ripetizioni complessive, per un totale di 25 cicli indipendenti di addestramento e test per ciascun algoritmo. Questo protocollo campiona l'intero spazio delle istanze salvaguardando la stima dell'errore da fluttuazioni causate dal *seed* di partizionamento.

Il modulo `main.py` esegue una comparazione sistematica a runtime tra le tre filosofie computazionali introdotte (Ensemble basato su Entropia, Margini geometrici in Spazi di Hilbert, e Modelli Connessionisti a Gradiente), valutando la media e la deviazione standard delle metriche di *Accuracy* e *F1-Score (Macro)*.

```
rkf = RepeatedKFold(n_splits=5, n_repeats=5, random_state=42)
```

```
scoring = {
```

```
    'accuracy': make_scorer(accuracy_score),
```

```
    'f1': make_scorer(f1_score, average='macro', zero_division=0)
```

```
}
```

```
# ...

for name, clf in models.items():

scores = cross_validate(clf, X_scaled, y, scoring=scoring, cv=rkf, n_jobs=-1)
```

4.8 L'Agente Adattativo: Apprendimento Online e Feedback Loop

Un sistema intelligente non è un artefatto statico, ma deve esibire flessibilità rispetto ai cambiamenti dell'ambiente e apprendere dalla propria esperienza. Terminata la fase di addestramento massivo (Batch Learning) descritta, il modulo `main.py` lancia una shell interattiva che simula il deployment sul campo.

L'operatore inserisce i sintomi grezzi osservati in tempo reale. Il sistema esegue al volo il Semantic Lifting, interroga l'ontologia, costruisce il feature vector e formula una diagnosi probabilistica. Il passaggio cruciale avviene immediatamente dopo: il sistema richiede un *feedback* all'operatore sulla correttezza della diagnosi. In caso di conferma, l'istanza arricchita viene dinamicamente aggiunta alla base di conoscenza statistica in memoria e il modello viene ri-addestrato in modalità **Online Learning**. Questo ciclo di feedback costante fa sì che l'agente computazionale migliori iterativamente le proprie performance affinando i pesi della Random Forest, adattandosi a derivate concettuali (concept drift) o all'emergere di nuove configurazioni di guasto direttamente nell'ambiente di produzione.

```
print("\n--- Feedback & Online Learning ---")

feedback = input("Qual era il guasto effettivo? (es. GuastoTermico, premi Invio per saltare): ").strip()

if feedback:

    X_list.append(raw_features)

    y_list.append(feedback)

    X_new_scaled = scaler.fit_transform(np.array(X_list))

    model.fit(X_new_scaled, np.array(y_list))

    print(" Rete Neurale aggiornata con successo!")
```

Capitolo 5 – Conclusioni e Sviluppi Futuri

5.1 Sintesi dei Risultati Raggiunti

Il progetto presentato ha dimostrato con successo la fattibilità e l'efficacia di un approccio ibrido per la diagnostica avanzata di sistemi meccatronici complessi, quale una stampante 3D a deposizione di filamento fuso (FDM). L'integrazione del framework OntoBK ha permesso di colmare il divario semantico (*semantic gap*) tipico dei modelli di Machine Learning tradizionali.

Affidando la modellazione formale del dominio a un'ontologia OWL 2.0 (TBox e ABox) e delegando la materializzazione della conoscenza implicita al Reasoner logico HermiT, il sistema è stato in grado di astrarre sintomi grezzi in categorie diagnostiche ad alto livello concettuale.

Questa operazione di *Semantic Lifting* ha prodotto uno spazio delle feature arricchito e multimodale che, integrando la telemetria continua dei sensori fisici con i concetti logici e analizzando i pattern attraverso un'infrastruttura multi-algoritmica (Random Forest, Support Vector Machine e Multi-Layer Perceptron), ha garantito prestazioni robuste anche in presenza del 15% di rumore stocastico inserito a livello di dataset.

La rigorosa validazione statistica tramite *Repeated K-Fold Cross-Validation* ha infine certificato la stabilità architetturale, l'efficacia del confronto tra diversi paradigmi di apprendimento e la reale capacità di generalizzazione dell'agente computazionale..

5.2 Valutazione Critica dell'Architettura

Dal punto di vista dell'Ingegneria della Conoscenza, il software realizzato incarna appieno la definizione teorica di agente intelligente computazionale, muovendosi entro i vincoli fisici di una memoria finita e di capacità di osservazione specializzate:

Appropriatezza e Gestione dei Limiti: L'integrazione della memoria cache semantica nel modulo `semantic_reasoner.py` dimostra una netta consapevolezza dei limiti computazionali intrinseci del reasoning esatto a Logica Descrittiva (caratterizzato da una complessità EXPTIME-complete). Tale accorgimento ingegneristico ha permesso di mitigare l'overhead delle interrogazioni sul grafo, rendendo l'approccio perfettamente scalabile ed efficiente durante l'iterazione e il *Semantic Lifting* dei 1500 record del dataset in pochissimi secondi.

Apprendimento dall'Esperienza e Flessibilità: La shell interattiva per l'efficace gestione dell'apprendimento online dimostra che l'agente non si configura come un artefatto statico. Il sistema esibisce flessibilità rispetto ai cambiamenti dell'ambiente integrando il feedback umano in tempo reale; l'introduzione di un caso d'uso reale attiva la retropropagazione dell'errore (Backpropagation), aggiornando istantaneamente i pesi sinaptici della Rete Neurale (Multi-Layer Perceptron) per affinarne la funzione di valutazione probabilistica.

Indipendenza Logica e Manutenibilità: Mantenendo rigorosamente scissi la base di conoscenza formale (`ontology_core.py`), il motore inferenziale deduttivo (`semantic_reasoner.py`), lo strato di astrazione delle feature (`ontoBK_learning.py`) e l'ambiente di addestramento statistico (`main.py`), il KBS garantisce i più alti standard di manutenibilità. Un ingegnere del software può espandere la topologia hardware della

macchina o modificare le asserzioni della ABox senza dover alterare una singola riga della logica di classificazione induttiva dei modelli predittivi.

5.3 Sviluppi Futuri

Nonostante la robustezza del KBS implementato, l'architettura si presta a diverse evoluzioni tecnologiche per avvicinarsi a un deployment di livello industriale:

- **Integrazione Sensoriale IoT (Internet of Things):** Attualmente, l'assertional box (ABox) e il vettore dei sintomi grezzi vengono popolati tramite input testuale dell'operatore o script di simulazione. Uno sviluppo futuro prevede l'interfacciamento diretto tramite API con il firmware della stampante 3D (es. Marlin o Klipper). I dati telemetrici (termistori, assorbimento motori, endstop) verrebbero tradotti istantaneamente in asserzioni OWL, rendendo il ciclo di percezione dell'agente completamente autonomo.
- **Espansione Espressiva dell'Ontologia:** La TBox potrebbe essere estesa integrando ragionamento spaziale qualitativo (es. topologia RCC-8) o temporale (Allen's interval algebra). Questo permetterebbe ad HermiT di dedurre guasti basati non solo su interconnessioni hardware, ma sulla sequenza temporale dei sintomi (es. *Sintomo A è avvenuto "prima" del Sintomo B*).
- **Federated Learning:** In uno scenario di *print farm* (batterie di decine di stampanti 3D), le istanze individuali dell'agente potrebbero condividere gli aggiornamenti dei pesi della Random Forest senza scambiarsi i log grezzi, implementando un apprendimento cooperativo e distribuito basato sulla medesima Background Knowledge ontologica.